



รายงาน PS03: Process Manipulation and Monitoring

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัชช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	2
1 คำสั่ง Linux สำหรับจัดการและเฝ้าดู process และ job	3
1.1 ความแตกต่างระหว่างคำสั่ง top และ ps	3
1.1.1 หลักการของทั้งสองคำสั่ง	3
1.1.2 ผลการทดลอง: คำสั่ง ps	5
1.1.3 ผลการทดลอง: คำสั่ง top	7
1.1.4 อภิปรายผล	7
1.2 จุดประสงค์ของคำสั่ง nice และวิธี nice process	9
1.2.1 จุดประสงค์	9
1.2.2 วิธี nice process	9
1.2.3 Source code ของโปรแกรมที่ใช้ทดลอง	10
1.2.4 ผลการทดลอง	12
1.2.5 อภิปรายผล	14
1.3 การ kill process และ job	16
1.3.1 หลักการ	16
1.3.2 วิธีอ้างอิงเป้าหมายที่จะส่งสัญญาณ	17
1.3.3 Source code ของโปรแกรมที่ใช้ทดลอง	18
1.3.4 ผลการทดลอง	19
1.3.5 อภิปรายผล	21
1.4 สรุปข้อ 1	23

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11)
Kernel	6.6.87.2-microsoft-standard-WSL2 (ตรวจด้วย <code>uname -r</code>)
จำนวน CPU	22 logical CPUs (ตรวจด้วย <code>nproc</code>)
ชุดคำสั่งที่ใช้	procps-ng 4.0.4 (ps, top, kill, pgrep, pkill) util-linux (renice, taskset) psmisc (killall)
คอมไพเลอร์	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
คำสั่งคอมไพล์	gcc -Wall -Wextra -O0 -o q1_burn q1_burn.c gcc -Wall -Wextra -o q1_sigdemo q1_sigdemo.c
ผลการคอมไพล์	ผ่านทุกไฟล์ ไม่มี warning
วันที่เก็บผล	31 สิงหาคม 2569 เวลา 14:55 น.

หมายเหตุเรื่อง WSL2 WSL2 รัน Linux kernel จริง คำสั่ง ps, top, nice, renice และ kill จึงทำงานเหมือน Linux ปกติทุกประการ

หมายเหตุเรื่องการทดลอง nice เครื่องที่ใช้ทดลองมี 22 CPU ถ้าปล่อยให้โปรแกรมทดสอบวิ่งอิสระทุก process จะได้ CPU ของตัวเองคนละแกน และจะ *ไม่เห็น* ผลของค่า nice เลย เพราะไม่มีการแย่ง CPU กันเกิดขึ้นจริง การทดลองข้อ 1.2 จึงใช้คำสั่ง `taskset -c 0` บังคับให้ process ทดสอบทุกตัว ทำงานอยู่บน CPU หมายเลข 0 เพียงแกนเดียว เพื่อสร้างสถานการณ์แย่ง CPU ซึ่งเป็นเงื่อนไขเดียวที่ค่า nice จะมีความหมาย

ไฟล์ผลการทดลองดิบ ผลลัพธ์ทุกบรรทัดที่ยกมาในรายงานฉบับนี้ถูกเก็บไว้ที่ PS03/result/q1_verified.txt และสร้างซ้ำได้ด้วยสคริปต์ PS03/src/run_q1.sh

1 คำสั่ง Linux สำหรับจัดการและเฝ้าดู process และ job

ข้อนี้เป็นการศึกษาคำสั่งพื้นฐานสามกลุ่มที่ใช้ทำงานกับ process บนลินุกซ์ คือ กลุ่ม เฝ้าดู (ps, top) กลุ่ม ปรับลำดับความสำคัญ (nice, renice) และกลุ่ม ส่งสัญญาณควบคุม (kill, pkill, killall) โดยทุกหัวข้อมีการทดลองจริงประกอบ ไม่ได้อธิบายจากคู่มือเพียงอย่างเดียว

1.1 ความแตกต่างระหว่างคำสั่ง top และ ps

1.1.1 หลักการของทั้งสองคำสั่ง

ทั้ง ps และ top อ่านข้อมูลจากแหล่งเดียวกันคือระบบไฟล์เสมือน /proc ซึ่งเคอร์เนลเปิดให้อ่านสถานะของทุก process ได้ ความต่างจึงไม่ได้อยู่ที่ ข้อมูล แต่อยู่ที่ วิธีนำเสนอข้อมูล

- ps (process status) เป็นคำสั่งแบบ **one-shot snapshot** คือเดินอ่าน /proc หนึ่งรอบ พิมพ์ผลออกมา แล้วจบการทำงานทันที ได้ภาพนิ่งของระบบ ณ วินาทีที่สั่ง
- top (table of processes) เป็นโปรแกรมแบบ **interactive** ที่วนอ่าน /proc ซ้ำๆ ทุก ๆ ช่วงเวลา (ค่าตั้งต้น 3 วินาที) แล้ววาดหน้าจอใหม่ทับของเดิม จนกว่าผู้ใช้จะกด q เพื่อออก ได้ภาพเคลื่อนไหวของระบบตามเวลาจริง

Table 1: สรุปความแตกต่างระหว่าง ps และ top

ประเด็น	ps	top
รูปแบบการทำงาน	one-shot อ่านครั้งเดียวแล้วจบ	วน อ่าน ซ้ำ และ วาด หน้า จอ ใหม่ เรื่อย ๆ
ความหมายของ %CPU	ค่าเฉลี่ยตลอดอายุของ process (เวลา CPU ที่ใช้ไปทั้งหมดหารด้วยอายุ)	ค่า เฉพาะ ช่วง รีเฟรช รอบ ล่าสุด เท่านั้น จึง สะท้อน สถานการณ์ ปัจจุบัน
ภาพรวมของระบบ	ไม่มี ต้องหาเอง เช่น ใช้ <code>wc -l</code> นับจำนวน process	มีให้ในส่วนหัว ทั้ง load average, จำนวน task แยกตามสถานะ, %Cpu(s) และหน่วยความจำ
การโต้ตอบ	ไม่มี สั่งอย่างใดได้ผลอย่างนั้น	มี กด k เพื่อ kill, r เพื่อ renice, M/P เพื่อเรียงตาม memory/CPU, 1 เพื่อแยกดูรายแกน
การนำผลไปใช้ต่อ	เหมาะสม ต่อ pipe เข้ากับ <code>grep</code> , <code>sort</code> , <code>awk</code> หรือเขียนลงไฟล์ในสคริปต์ได้เลย	ไม่เหมาะ เพราะผลลัพธ์เป็นหน้าจอ ต้องสั่ง <code>top -b -n 1</code> (batch mode) จึงจะนำไปใช้ต่อได้
การเลือกคอลัมน์	ยืดหยุ่นสูงมากด้วย <code>-o</code> และเรียงลำดับด้วย <code>--sort</code>	เลือก ได้ เฉพาะ ชุด คอลัมน์ ที่ โปรแกรมเตรียมไว้
ภาระต่อระบบ	แทบไม่มี เพราะจบทันที (วัดได้ 0.00 วินาที)	มีเล็กน้อยตลอดเวลาที่เปิดค้างไว้ เพราะต้องอ่าน <code>/proc</code> ซ้ำๆ ทั่วรอบ
สถานการณ์ที่เหมาะสม	ต้องการ ค้นหา process หนึ่งตัว หรือเก็บข้อมูลเข้าสคริปต์	ต้องการ เฝ้าดู ว่าตอนนี้ใครกิน CPU หรือ RAM มากที่สุด

1.1.2 ผลการทดลอง: คำสั่ง ps

เมื่อสั่ง ps เปล่า ๆ จะเห็นเฉพาะ process ที่เป็นของผู้ใช้คนเดียวกัน และผูกอยู่กับ terminal (pts/4) เดียวกันเท่านั้น ไม่ใช่ทั้งระบบ

```
$ ps
  PID TTY          TIME CMD
 5688 pts/4    00:00:00 bash
 5689 pts/4    00:00:00 bash
 5691 pts/4    00:00:00 sed
 5744 pts/4    00:00:00 ps
```

Code 1: ps แบบไม่ใส่ option

การจะเห็นทั้งระบบต้องใส่ option เพิ่ม ซึ่ง ps รองรับไวยากรณ์สองแบบ คือแบบ BSD (aux ไม่มีชี้คหน้าหน้า) และแบบ UNIX (-ef มีชี้คหน้าหน้า)

```
$ ps aux | head -8
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1  0.0  0.0  21792 12084 ?        Ss   12:03   0:01 /sbin/init
root             2  0.0  0.0   3072  1760 ?        Sl   12:03   0:00 /init
root             7  0.0  0.0   3132  2112 ?        Sl   12:03   0:07 plan9
--control-socket 7 ...
root            56  0.0  0.0 66828 14196 ?        S<s  12:03   0:01
/usr/lib/systemd/systemd-journald
root           102  0.0  0.0  25016  5984 ?        Ss   12:03   0:06
/usr/lib/systemd/systemd-udev
systemd+       152  0.0  0.0  21336 12672 ?        Ss   12:03   0:00
/usr/lib/systemd/systemd-resolved
systemd+       161  0.0  0.0  91028  7568 ?        Ssl  12:03   0:00
/usr/lib/systemd/systemd-timesyncd
```

Code 2: ps aux – ไวยากรณ์แบบ BSD

```
$ ps -ef | head -8
UID          PID    PPID  C STIME TTY          TIME CMD
root             1        0  0 12:03 ?          00:00:01 /sbin/init
root             2        1  0 12:03 ?          00:00:00 /init
root             7        2  0 12:03 ?          00:00:07 plan9 --control-socket 7 ...
root            56        1  0 12:03 ?          00:00:01 /usr/lib/systemd/systemd-journald
root           102        1  0 12:03 ?          00:00:06 /usr/lib/systemd/systemd-udev
systemd+       152        1  0 12:03 ?          00:00:00 /usr/lib/systemd/systemd-resolved
systemd+       161        1  0 12:03 ?          00:00:00 /usr/lib/systemd/systemd-timesyncd
```

Code 3: ps -ef – ไวยากรณ์แบบ UNIX มีคอลัมน์ PPID

ความต่างที่ใช้งานจริงคือ ps aux ให้ %CPU, %MEM และ STAT มาด้วย ส่วน ps -ef ให้คอลัมน์ PPID ซึ่งจำเป็นเวลาต้องไล่ความสัมพันธ์พ่อลูกของ process (เช่นที่ใช้ตรวจ orphan process ใน PS02)

จุดแข็งที่สุดของ ps คือ option -o ที่ให้เลือกคอลัมน์เองได้ และ --sort ที่เรียงลำดับได้ ในการทดลองนี้ได้สั่งโปรแกรม q1_burn ให้กิน CPU ค้างไว้เป็นเป้าก่อน แล้วจึงเรียงตาม %CPU จากมากไปน้อย

```
$ taskset -c 0 ./q1_burn 30 MONITOR_ME & # a CPU-hungry process to watch
$ ps -eo pid,ppid,ni,pri,stat,%cpu,%mem,etime,comm --sort=-%cpu | head -6
  PID    PPID  NI  PRI  STAT %CPU %MEM     ELAPSED COMMAND
 5749    5689   0   19  R+    109  0.0         00:02 q1_burn
```

```

    7      2    0   19 Sl    0.0  0.0    02:51:41 init
   102     1    0   19 Ss    0.0  0.0    02:51:41 systemd-udevd
  5683   102    0   19 S     0.0  0.0      01:46 (udev-worker)
    1      0    0   19 Ss    0.0  0.0    02:51:42 systemd

```

Code 4: `ps -eo ... --sort=-%cpu` เลือกคอลัมน์เองและเรียงลำดับ

และเนื่องจาก `ps` จบการทำงานทันที จึงวัดเวลาได้ว่าใช้เวลาน้อยกว่า 0.01 วินาที

```

$ /usr/bin/time -f "real %e s" ps aux > /dev/null
real 0.00 s
$ ps -e --no-headers | wc -l
39

```

Code 5: `ps` เป็นภาพนิ่งครั้งเดียวจึงจบทันที

1.1.3 ผลการทดลอง: คำสั่ง top

โดยปกติ top จะยืดหน้าจอไว้และวาดใหม่เรื่อย ๆ ซึ่งเก็บลงรายงานไม่ได้ จึงต้องใช้ option -b (batch mode) คู่กับ -n เพื่อกำหนดจำนวนรอบ

```
$ top -b -n 1 | head -13
top - 14:55:11 up 2:51, 2 users, load average: 0.10, 0.24, 0.30
Tasks: 39 total, 2 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.5 us, 0.0 sy, 0.0 ni, 95.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15716.6 total, 13911.9 free, 1368.4 used, 694.1 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used, 14348.3 avail Mem

    PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  5749 poomipat  20   0   2548   1056   1056 R   100.0   0.0   0:02.43 q1_burn
      1 root      20   0   21792  12084   9092 S    0.0   0.1   0:01.89 systemd
      2 root      20   0    3072   1760   1760 S    0.0   0.0   0:00.10 init-systemd(Ub
      7 root      20   0    3132   2112   1760 S    0.0   0.0   0:07.89 init
     56 root      19  -1   66828  14196  13492 S    0.0   0.1   0:01.41 systemd-journal
    102 root      20   0   25016   5984   4928 S    0.0   0.0   0:06.86 systemd-udevd
```

Code 6: top -b -n 1 – ถ่ายภาพหนึ่งรอบ

ห้บรรทัดแรกคือ **ส่วนสรุปภาพรวมของระบบ** ซึ่ง ps ไม่มีให้เลย ได้แก่ เวลาที่ระบบเปิดมาแล้ว, load average 1/5/15 นาที, จำนวน task แยกตามสถานะ (running / sleeping / stopped / zombie), สัดส่วนการใช้ CPU แยกเป็น user, system, nice, idle, iowait และปริมาณหน่วยความจำกับ swap

เพื่อยืนยันว่า top คำนวณ %CPU ใหม่ทุกรอบจริง ได้สั่งให้ถ่ายสองภาพ ห่างกัน 1 วินาที แล้วกรองเฉพาะบรรทัดที่สนใจ

```
$ top -b -n 2 -d 1 | grep -E "^top -|^Tasks|^%Cpu|q1_burn"
top - 14:55:11 up 2:51, 2 users, load average: 0.10, 0.24, 0.30
Tasks: 39 total, 2 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.4 us, 0.0 sy, 0.0 ni, 95.6 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
  5749 poomipat  20   0   2548   1056   1056 R   100.0   0.0   0:02.66 q1_burn
top - 14:55:12 up 2:51, 2 users, load average: 0.10, 0.24, 0.30
Tasks: 39 total, 2 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.5 us, 0.0 sy, 0.0 ni, 95.4 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
  5749 poomipat  20   0   2548   1056   1056 R   100.0   0.0   0:03.76 q1_burn
```

Code 7: top -b -n 2 -d 1 – สองภาพห่างกัน 1 วินาที

สังเกตว่าเวลาที่ส่วนหัวเดินจาก 14:55:11 ไป 14:55:12 และ TIME+ ของ q1_burn เพิ่มขึ้นจาก 0:02.66 เป็น 0:03.76 คือเพิ่มขึ้น 1.10 วินาทีในเวลาจริง 1 วินาที ซึ่งตรงกับที่ %CPU รายงานว่า 100.0 พอดี ถ้าเป็น ps จะได้เพียงภาพเดียวและไม่มีทาง เห็นความเปลี่ยนแปลงนี้

1.1.4 อภิปรายผล

- ต่างกันที่ **มิติเวลา** เป็นหลัก: ps ตอบคำถามว่า “ณ วินาทีนี้มีอะไรอยู่ในระบบบ้าง” ส่วน top ตอบคำถามว่า “ตอนนี้ระบบกำลังทำอะไรอยู่” การทดลองยืนยันข้อนี้ชัดเจน คือ ps aux ใช้เวลา 0.00 วินาที แล้วจบ ในขณะที่ top ต้องถูกบังคับด้วย -n 1 ถึงจะยอมจบ ถ้าไม่ใส่จะรันค้างไปเรื่อย ๆ
- ค่า %CPU ของสองคำสั่งนี้ “ไม่ใช่ค่าเดียวกัน”: เป็นจุดที่ทำให้สับสนบ่อยที่สุด ค่าของ ps คือ *เวลา CPU ที่ใช้ไปทั้งหมดหารด้วยอายุของ process* จึงเป็นค่าเฉลี่ย ย้อนหลังตลอดอายุ ส่วนค่าของ top คัดจาก

ส่วนต่างของเวลา CPU ระหว่างสองรอบรีเฟรชล่าสุด จึงเป็นค่าปัจจุบัน ผลที่ตามมาคือ process ที่เคยกิน CPU หนักเมื่อชั่วโมงที่แล้วแต่ตอนนี้ หลับอยู่ จะยังมี %CPU สูงใน ps แต่เป็น 0.0 ใน top ในการทดลองนี้ ps รายงาน 109 ให้ q1_burn ที่เพิ่งเกิดได้ 2 วินาที (คอลัมน์ ELAPSED แสดง 00:02 ซึ่งปัดเป็นวินาทีเต็ม จึงหารแล้วเกิน 100 ได้เล็กน้อย) ขณะที่ top รายงาน 100.0 ตรงตามความเป็นจริง

- **top** ให้ภาพรวมของ “ระบบ” ส่วน **ps** ให้ภาพของ “process”: ข้อมูล load average, จำนวน zombie, %Cpu(s) แยก us/sy/ni/id/wa และหน่วยความจำรวม มีเฉพาะใน top ถ้าจะเอาข้อมูลชุดนี้จาก ps ต้องไปอ่านจาก /proc/loadavg และ /proc/meminfo เอง
- **ps** ขนะขาดเรื่องการนำผลไปใช้ต่อ: เพราะผลลัพธ์เป็นข้อความบรรทัดต่อบรรทัดที่ต่อ pipe ได้ทันที และเลือกคอลัมน์เองได้ด้วย -o ดังที่ใช้ในการทดลองข้อ 1.2 และ 1.3 ตลอดทั้งรายงานนี้ ส่วน top ต้องพึ่ง -b เพื่อบังคับให้พิมพ์ เป็นข้อความธรรมดา ซึ่งก็ยังคง grep ตัดส่วนหัวออกอยู่ดี
- **top** ทำสิ่งที่ **ps** ทำไม่ได้ คือ “สั่งการ”: ps เป็นคำสั่งอ่านอย่างเดียว แต่ใน top กด k เพื่อส่ง signal และกด r เพื่อ renice ได้ทันทีโดยไม่ต้องออกจากโปรแกรม จึงเป็นเครื่องมือที่ครบวงจรกว่าเมื่อต้องแก้ปัญหาเฉพาะหน้า เช่นเจอ process ที่กิน CPU ผิดปกติแล้วต้องจัดการเดี๋ยวนั้น

1.2 จุดประสงค์ของคำสั่ง nice และวิธี nice process

1.2.1 จุดประสงค์

`nice` คือคำสั่งที่ใช้ บอกใบ้ (hint) ให้ scheduler ทราบว่า process ตัวนี้สำคัญมากหรือน้อยเพียงใด เมื่อเทียบกับ process อื่นที่กำลังแย่ง CPU กันอยู่ โดยกำหนดผ่านตัวเลขที่เรียกว่า *nice value* หรือ *niceness*

- ช่วงค่าคือ -20 ถึง $+19$ ค่าตั้งต้นของทุก process คือ 0
- ค่า ยิ่งมาก ยิ่งใจดี คือยอมสละ CPU ให้คนอื่นใช้ก่อน ตัวเองจึงได้ priority ต่ำ
- ค่า ยิ่งน้อย (ติดลบ) ยิ่งเห็นแก่ตัว คือแย่ง CPU มาใช้ก่อน ตัวเองจึงได้ priority สูง

จุดที่ต้องเน้นคือ `nice` ไม่ใช้การจองหรือจำกัดโควตา CPU ถ้าไม่มีใครแย่ง CPU อยู่ process ที่ `nice` เป็น 19 ก็ยังใช้ CPU ได้เต็ม 100% ค่า `nice` จะมีผลก็ต่อเมื่อเกิดการแย่งกันแย่ง CPU ขึ้นจริงเท่านั้น (ข้อสรุปนี้พิสูจน์ด้วยตัวเลขในหัวข้ออภิปรายผล)

การใช้งานที่พบบ่อยคือสิ่งงานหนักที่ “ไม่รีบ” เช่นการ backup, การ compile โปรแกรมใหญ่ หรือการ encode วิดีโอ ให้ทำงานด้วยค่า `nice` สูง ๆ เพื่อให้งานเหล่านั้นใช้ CPU เฉพาะตอนที่เครื่องว่าง และไม่ไปแย่ง CPU จนหน้าจอผู้ใช้กระตุก

1.2.2 วิธี nice process

มีอยู่สองกรณีตามจังหวะเวลา

กรณีที่ 1: กำหนดตั้งแต่ตอนเริ่มโปรแกรม ใช้คำสั่ง `nice`

```
$ nice -n <value> <command> [arguments...]

$ nice -n 19 ./q1_burn 15 LOW      # start the program with nice = 19
$ nice ./q1_burn 15 LOW           # without -n the default is nice = 10
```

Code 8: รูปแบบการใช้ `nice`

กรณีที่ 2: เปลี่ยนค่าของ process ที่รันอยู่แล้ว ใช้คำสั่ง `renice`

```
$ renice -n <value> -p <PID>      # one process at a time
$ renice -n <value> -u <username> # every process owned by that user
$ renice -n <value> -g <PGID>     # every process in that process group
```

Code 9: รูปแบบการใช้ `renice`

นอกจากนี้ยังเปลี่ยนได้จากหน้าจอ `top` โดยกดปุ่ม `r` แล้วพิมพ์ PID กับค่าใหม่ ซึ่งเบื้องหลังก็คือการเรียก system call `setpriority()` เหมือนกัน

ข้อจำกัดด้านสิทธิ์ ผู้ใช้ทั่วไป เพิ่ม ค่า `nice` ได้เสมอ (ยอมเสียเปรียบตัวเอง) แต่ ลด ค่าไม่ได้เลย แม้จะเป็นการลด กลับมาที่ค่าเดิมของ process ตัวเอง การลดค่า `nice` ต้องเป็น `root` หรือมีสิทธิ์ `CAP_SYS_NICE` เท่านั้น เหตุผลคือถ้าใครก็ตั้งค่าติดลบได้ ทุกคนก็จะตั้งเป็น -20 กันหมด กลไกนี้ก็จะไร้ความหมาย

1.2.3 Source code ของโปรแกรมที่ใช้ทดลอง

เพื่อวัดผลของ nice เป็นตัวเลข ได้เขียนโปรแกรม q1_burn.c ที่วนคำนวณกิน CPU ตามเวลาที่กำหนด แล้วรายงานว่าทำงานได้กี่ “รอบ” ถ้ารัน 2 process พร้อมกัน อัตราส่วนของจำนวนรอบก็คืออัตราส่วนของ CPU ที่ scheduler แบ่งให้แต่ละตัวโดยตรง

```

1  /* CPE 333 PS03 - Q1
2  * CPU burner used to demonstrate nice / renice / kill.
3  * Usage: ./q1_burn <seconds> <label>
4  *
5  * It spins for <seconds> of wall clock time and reports how many rounds of
6  * work it managed to finish. Two burners started together therefore report
7  * numbers that can be compared directly: the ratio of the two round counts
8  * is the ratio of the CPU share the scheduler gave them.
9  */
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <unistd.h>
13 #include <sys/resource.h>
14 #include <sys/time.h>
15
16 #define WORK_PER_ROUND 200000
17
18 int main(int argc, char *argv[])
19 {
20     int seconds = (argc > 1) ? atoi(argv[1]) : 10;
21     const char *label = (argc > 2) ? argv[2] : "burn";
22
23     struct timeval start, now;
24     gettimeofday(&start, NULL);
25
26     unsigned long long rounds = 0;
27     double sum = 0.0;
28     double elapsed;
29
30     for (;;) {
31         for (int i = 0; i < WORK_PER_ROUND; i++) /* one round of pure CPU work */
32             sum += i * 0.5;
33         rounds++;
34
35         gettimeofday(&now, NULL);
36         elapsed = (now.tv_sec - start.tv_sec)
37                 + (now.tv_usec - start.tv_usec) / 1e6;
38         if (elapsed >= seconds)
39             break;
40     }
41
42     int ni = getpriority(PRIO_PROCESS, 0); /* the nice value we are running at */
43
44     (void)sum; /* compiled with -O0, so the work loop is never optimized away */
45
46     printf("[%s] pid=%d nice=%d rounds=%llu wall=%.2fs\n",
47            label, getpid(), ni, rounds, elapsed);
48     fflush(stdout);
49     return 0;
50 }

```

Code 10: q1_burn.c – CPU burner สำหรับวัดผลของ nice

คำอธิบายการทำงาน: โปรแกรมวนลูปคำนวณเลขทศนิยม `WORK_PER_ROUND` ครั้งนับเป็นหนึ่งรอบ แล้ว
ใช้คณานุกรมด้วย `gettimeofday()` ว่าครบเวลาที่สั่ง หรือยัง เมื่อครบจึงอ่านค่า `nice` ของตัวเองด้วย `getpriority()`
แล้วพิมพ์จำนวนรอบที่ทำได้ออกมา คอมไพล์ด้วย `-O0` เพื่อไม่ให้คอมไพเลอร์ ตัดลูปคำนวณทิ้ง

1.2.4 ผลการทดลอง

การทดลองที่ 1: ค่า nice ตั้งต้นและการสั่ง nice ใช้วิธีเรียก nice ซ้อนตัวเอง เพราะ nice ที่ไม่มีอาร์กิวเมนต์ จะพิมพ์ค่า nice ปัจจุบันออกมา

```
$ nice -> 0
$ nice -n 10 nice -> 10
$ nice -n 19 nice -> 19
$ nice -n 5 nice -n 5 nice (increments add up) -> 10
```

Code 11: ค่า nice ตั้งต้นของ process ใหม่

บรรทัดสุดท้ายแสดงคุณสมบัติสำคัญคือ nice ทำงานแบบ **เพิ่มค่าสะสม** (increment) ไม่ใช่การกำหนดค่าตายตัว การเรียกซ้อนกันสองชั้นชั้นละ 5 จึงได้ผลเป็น 10

การทดลองที่ 2 (กรณีควบคุม): burner สองตัว ค่า nice เท่ากัน ผู้ไวก่ CPU 0 ทั้งคู่ เพื่อพิสูจน์ว่าเมื่อ nice เท่ากัน scheduler จะแบ่ง CPU ให้เท่า ๆ กัน

```
$ taskset -c 0 ./q1_burn 15 A_nice0 &
$ taskset -c 0 ./q1_burn 15 B_nice0 &
$ ps -o pid,ni,pri,psr,stat,%cpu,time,args -p <pid1>,<pid2> # while both are
  running
  PID NI PRI PSR STAT %CPU    TIME COMMAND
  5765  0  19  0 R+   55.3 00:00:02 /tmp/ps03_q1/q1_burn 15 A_nice0
  5766  0  19  0 R+   55.5 00:00:02 /tmp/ps03_q1/q1_burn 15 B_nice0

[A_nice0] pid=5765 nice=0 rounds=19068 wall=15.00s
[B_nice0] pid=5766 nice=0 rounds=19119 wall=15.00s
```

Code 12: กรณีควบคุม: nice 0 เทียบกับ nice 0

การทดลองที่ 3 (กรณีทดลอง): burner สองตัว ค่า nice ต่างกัน ตัวหนึ่ง nice เป็น 0 อีกตัว nice เป็น 19 เงื่อนไขอื่นเหมือนเดิมทุกอย่าง

```
$ taskset -c 0 ./q1_burn 15 HIGH_nice0 &
$ taskset -c 0 nice -n 19 ./q1_burn 15 LOW_nice19 &
$ ps -o pid,ni,pri,psr,stat,%cpu,time,args -p <pid1>,<pid2> # while both are
  running
  PID NI PRI PSR STAT %CPU    TIME COMMAND
  5771  0  19  0 R+   106 00:00:04 /tmp/ps03_q1/q1_burn 15 HIGH_nice0
  5772 19  0  0 RN+   1.7 00:00:00 /tmp/ps03_q1/q1_burn 15 LOW_nice19

$ top -b -n 1 -p <pid1> -p <pid2>

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
  5771 poomipat   20   0   2548   1056   1056 R  100.0   0.0   0:04.48 q1_burn
  5772 poomipat   39  19   2548   1056   1056 R    0.0   0.0   0:00.07 q1_burn

[HIGH_nice0] pid=5771 nice=0 rounds=29924 wall=15.00s
[LOW_nice19] pid=5772 nice=19 rounds=465 wall=15.00s
```

Code 13: กรณีทดลอง: nice 0 เทียบกับ nice 19

การทดลองที่ 4: renice process ที่กำลังรันอยู่

```
$ ps -o pid,ni,pri,stat,comm -p <pid>          # before renice
  PID  NI PRI STAT COMMAND
  5777   0 19 R+  q1_burn
$ renice -n 15 -p 5777
5777 (process ID) old priority 0, new priority 15
$ ps -o pid,ni,pri,stat,comm -p <pid>          # after renice
  PID  NI PRI STAT COMMAND
  5777  15  4 RN+  q1_burn

[renice_target] pid=5777 nice=15 rounds=30658 wall=12.00s
```

Code 14: renice ระหว่างที่โปรแกรมกำลังทำงาน

การทดลองที่ 5: ข้อจำกัดด้านสิทธิ์

```
current user : poomipat (uid=1000)
$ nice -n -5 nice
nice: cannot set niceness: Permission denied
0                                <-- the command still runs, at the unchanged nice value

$ sleep 60 &                      -> pid 5785
$ renice -n -5 -p 5785
renice: failed to set priority for 5785 (process ID): Permission denied
exit status = 1

$ renice -n +8 -p 5785              # raising the nice value is always allowed
5785 (process ID) old priority 0, new priority 8
  PID  NI PRI STAT COMMAND
  5785   8 11 SN+  sleep

$ renice -n 0 -p 5785              # lowering it back is refused, even for our own process
renice: failed to set priority for 5785 (process ID): Permission denied
exit status = 1
```

Code 15: ผู้ใช้ทั่วไปลดค่า nice ไม่ได้

1.2.5 อภิปรายผล

Table 2: สรุปผลการทดลองผลของค่า nice ต่อการแบ่ง CPU (ทุกกรณีถูกผูกไว้ที่ CPU 0)

การทดลอง	nice	รอบที่ทำได้	รอบ/วินาที	สัดส่วน
ควบคุม: A_nice0	0	19,068	1,271	1.00
ควบคุม: B_nice0	0	19,119	1,275	1.00
ทดลอง: HIGH_nice0	0	29,924	1,995	64.4
ทดลอง: LOW_nice19	19	465	31	1.00
รันตัวเดียว: renice_target	15	30,658	2,555	—

- ค่า nice เท่ากัน scheduler แบ่ง CPU ให้เท่ากันจริง: กรณีควบคุมได้ 19,068 กับ 19,119 รอบ ต่างกันเพียง 0.27% ซึ่งถือว่าเท่ากันในทางปฏิบัติ สอดคล้องกับที่ ps รายงาน %CPU ให้ทั้งคู่ที่ 55.3 กับ 55.5 การมีกรณีควบคุมนี้ทำให้มั่นใจได้ว่าความต่างที่เห็นในกรณีทดลอง เกิดจากค่า nice จริง ๆ ไม่ใช่ความบังเอิญของเครื่อง
- ค่า nice ต่างกันสร้างความต่างมหาศาล: เมื่อเปลี่ยนเพียงค่า nice ของตัวหนึ่งเป็น 19 ผลคือ 29,924 รอบ เทียบกับ 465 รอบ หรือ ประมาณ 64 เท่า ตัวเลขนี้ยืนยันว่า nice มีผลจริงและมีผลมาก ค่า %CPU ที่ ps รายงาน (106 เทียบกับ 1.7) และ TIME+ ที่ top รายงาน (0:04.48 เทียบกับ 0:00.07) ก็ชี้ไปทางเดียวกัน
- nice มีความหมายเฉพาะตอนที่มีการแย่ง CPU เท่านั้น: เป็นข้อสรุปที่ตัวเลขในตารางแสดงชัดที่สุด process renice_target ถูกตั้งเป็น nice 15 ซึ่งถือว่า “ใจดี” มาก แต่เพราะรันอยู่ตัวเดียวไม่มีใครแย่ง จึงทำได้ถึง 2,555 รอบต่อวินาที ซึ่ง สูงที่สุดในทุกการทดลอง สูงกว่า HIGH_nice0 ที่มีค่า nice เป็น 0 เสียอีก ข้อนี้ตอกย้ำว่า nice ไม่ใช่การจำกัดโควตา แต่เป็นการกำหนด ลำดับ ในการแย่งเท่านั้น ถ้า CPU ยังเหลือ ทุกคนก็ได้ใช้เต็มที่ไม่ว่าค่า nice จะเป็นเท่าใด
- CPU หนึ่งแกนถูกใช้งานเต็มในทุกกรณี ต่างกันแค่วิธีแบ่ง: เมื่อรวมผลงานของสอง process ในกรณีควบคุมเข้าด้วยกันจะได้ $1,271 + 1,275 = 2,546$ รอบต่อวินาที ซึ่งเท่ากับกรณีที่มี process เดียวครองแกนนั้นทั้งแกน (2,555 รอบต่อวินาที) โดยต่างกันเพียง 0.4% แปลว่า scheduler ไม่ได้ทำให้งานรวมลดลง เพียงแต่แบ่งเค้กก่อนเดิมออกเป็นสองส่วนเท่า ๆ กัน ส่วนในกรณีทดลองก็เป็นเค้กก้อนเดิม เพียงแต่ถูกแบ่งเป็น 64 ต่อ 1 ตามค่า nice ที่ตั้งไว้
- ความสัมพันธ์ระหว่างคอลัมน์ NI, PRI และ PR: จากผลการทดลองสรุปได้ว่าคอลัมน์ PRI ของ ps มีค่าเท่ากับ $19 - NI$ (nice 0 ได้ PRI 19, nice 15 ได้ PRI 4, nice 19 ได้ PRI 0 และ systemd-journald ที่ nice -1 ได้ PRI 20) ส่วนคอลัมน์ PR ของ top มีค่าเท่ากับ $20 + NI$ (nice 0 ได้ PR 20, nice 19 ได้ PR 39) นั่นคือทั้งสองค่าแสดงข้อมูล ชัดเดียวกันแต่ “กลับด้าน” กัน ของ ps ยิ่งมากยิ่งขึ้นสำคัญ ส่วนของ top เป็นเลข priority ดิบของเคอร์เนลซึ่งยิ่งน้อยยิ่งสำคัญ เวลาอ่านรายงานจึงต้องดูให้ติว่ากำลังอ่านคอลัมน์ของค่าสิ่งไหน
- คอลัมน์ STAT มีตัวอักษร N เพิ่มเข้ามา: สังเกตว่า process ที่ nice มากกว่า 0 จะมีสถานะเป็น RN+ หรือ SN+ ตัว N คือเครื่องหมายบอกว่า “process นี้มีค่า nice เป็นบวก” ซึ่งใช้เป็นวิธีตรวจสอบอย่างรวดเร็วได้ว่า มีใครถูก renice ไว้บ้าง

- สิทธิ์เป็นแบบทางเดียว: การทดลองที่ 5 แสดงว่าผู้ใช้ทั่วไป “สละสิทธิ์” ได้เสมอ (`renice +8` สำเร็จ) แต่จะ “ทวงคืน” ไม่ได้เลย (`renice 0` กลับล้มเหลว ทั้งที่เป็น process ของตัวเองแท้ ๆ) พฤติกรรมนี้เป็นไปตามที่ระบุใน `man 2 setpriority` และเป็นการป้องกัน ไม่ให้ผู้ใช้คนใดคนหนึ่งตั้งค่าติดลบเพื่อยึด CPU ทั้งเครื่องไว้เอง อีกจุดที่น่าสนใจคือคำสั่ง `nice -n -5 nice` แม้จะตั้งค่าไม่สำเร็จ แต่ยัง รันโปรแกรมต่อ ด้วยค่า `nice` เดิม (พิมพ์ 0 ออกมา) ไม่ได้ยกเลิกการทำงานทิ้ง

1.3 การ kill process และ job

1.3.1 หลักการ

ชื่อคำสั่ง `kill` ทำให้เข้าใจผิดได้ง่าย เพราะหน้าที่จริงของมันคือ **ส่งสัญญาณ (signal)** ไปให้ **process** ไม่ใช่การฆ่าเสมอไป สัญญาณบางตัวสั่งให้หยุดชั่วคราว บางตัวสั่งให้ทำงานต่อ บางตัวเพียงแค่แจ้งเตือน ที่คนเข้าใจว่า `kill` คือการฆ่า เป็นเพราะสัญญาณ **ดั่งต้น** ที่ส่งไป คือ `SIGTERM` ซึ่งมีพฤติกรรมโดยปริยายคือให้ process จบการทำงาน

```
$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN  ...           63) SIGRTMAX-1 64) SIGRTMAX
```

Code 16: `kill -l` แสดงรายชื่อสัญญาณทั้งหมด (ตัดมาบางส่วน)

Table 3: สัญญาณที่ใช้บ่อยในการจัดการ process

เลข	ชื่อ	พฤติกรรมโดยปริยาย	ดักจับได้หรือไม่
1	SIGHUP	จบการทำงาน ใช้เมื่อ terminal ถูกปิด แต่ daemon หลายตัวตีความว่า “โหลด config ใหม่”	ได้
2	SIGINT	จบการทำงาน คือสัญญาณที่เกิดจากการกด <code>CTRL+C</code>	ได้
3	SIGQUIT	จบการทำงานพร้อมสร้าง core dump (<code>CTRL+\</code>)	ได้
9	SIGKILL	จบการทำงานทันที เคอร์เนลจัดการให้เอง	ไม่ได้
15	SIGTERM	ขอให้จบการทำงาน (ค่าดั่งต้นของ <code>kill</code>)	ได้
18	SIGCONT	สั่งให้ process ที่หยุดอยู่ทำงานต่อ	ได้
19	SIGSTOP	สั่งให้หยุดชั่วคราว	ไม่ได้
20	SIGTSTP	หยุดชั่วคราว คือสัญญาณที่เกิดจากการกด <code>CTRL+Z</code>	ได้

1.3.2 วิธีอ้างถึงเป้าหมายที่จะส่งสัญญาณ

Table 4: วิธีระบุเป้าหมายของคำสั่ง `kill` และคำสั่งในตระกูลเดียวกัน

รูปแบบคำสั่ง	ความหมาย
<code>kill 1234</code>	ส่ง SIGTERM ให้ process หมายเลข 1234
<code>kill -9 1234</code>	ส่ง SIGKILL (ระบุด้วยเลข)
<code>kill -KILL 1234</code>	ส่ง SIGKILL (ระบุด้วยชื่อ ให้ผลเหมือนกัน)
<code>kill %1</code>	ส่ง SIGTERM ให้ job หมายเลข 1 ของ shell ปัจจุบัน
<code>kill -- -1234</code>	ส่งให้ทุก process ใน process group หมายเลข 1234
<code>pkill -f "pattern"</code>	ส่งให้ทุก process ที่ <i>command line</i> เดิม ตรงกับรูปแบบ
<code>killall progname</code>	ส่งให้ทุก process ที่ ชื่อโปรแกรม ตรงกันแบบตรงตัว
<code>kill -0 1234</code>	ไม่ส่งสัญญาณจริง แต่ตรวจว่า PID นั้นยังอยู่และเรามีสิทธิ์ส่งหรือไม่

1.3.3 Source code ของโปรแกรมที่ใช้ทดลอง

เพื่อพิสูจน์ว่า SIGTERM ดักจับได้แต่ SIGKILL ดักจับไม่ได้ ได้เขียนโปรแกรม q1_sigdemo.c ที่ “ดี” คือ ดักจับ SIGTERM และ SIGINT ไว้แล้วไม่ยอมตาย

```

1  /* CPE 333 PS03 - Q1
2  * A process that catches SIGTERM and SIGINT and refuses to exit,
3  * used to show why SIGKILL (kill -9) exists.
4  */
5  #include <stdio.h>
6  #include <signal.h>
7  #include <unistd.h>
8
9  static const char msg_term[] = "[sigdemo] caught SIGTERM (15) -- handler ran, still
    alive\n";
10 static const char msg_int[] = "[sigdemo] caught SIGINT (2) -- handler ran, still
    alive\n";
11
12 static void handler(int sig)
13 {
14     /* write() is async-signal-safe, printf() is not */
15     if (sig == SIGTERM)
16         write(STDOUT_FILENO, msg_term, sizeof msg_term - 1);
17     else
18         write(STDOUT_FILENO, msg_int, sizeof msg_int - 1);
19 }
20
21 int main(void)
22 {
23     signal(SIGTERM, handler);
24     signal(SIGINT, handler);
25
26     printf("[sigdemo] pid = %d, catching SIGTERM(15) and SIGINT(2)\n", getpid());
27     fflush(stdout);
28
29     for (;;)
30         pause();    /* sleep until a signal arrives */
31
32     return 0;
33 }

```

Code 17: q1_sigdemo.c – process ที่ดักจับสัญญาณและไม่ยอมจบการทำงาน

คำอธิบายการทำงาน: ลงทะเบียนฟังก์ชัน handler() ไว้กับ SIGTERM และ SIGINT ด้วย signal() จากนั้นวนเรียก pause() ซึ่งจะทำให้ process หลับรอจนกว่าจะมีสัญญาณเข้ามา เมื่อสัญญาณมาถึง handler() จะถูกเรียก พิมพ์ข้อความ แล้ว pause() ก็จะกลับมาวนรออีกครั้ง process จึงไม่มีวันจบด้วยสองสัญญาณนี้ ในตัว handler ใช้ write() แทน printf() เพราะ write() เป็น async-signal-safe ส่วน printf() ไม่ใช่ การเรียก printf() ใน handler ขณะที่โปรแกรมหลักกำลังใช้ buffer อยู่อาจทำให้ข้อมูลเสียหายได้

1.3.4 ผลการทดลอง

การทดลองที่ 1: kill เลข ๆ คือการส่ง SIGTERM

```
$ sleep 300 &          -> pid 5790
  PID    PPID STAT COMMAND
  5790    5689 S+   sleep
$ kill 5790
$ ps -o pid,stat,comm -p 5790      # no data row = the process is gone
  PID STAT COMMAND
exit status of ps = 1
```

Code 18: kill ด้วยสัญญาณตั้งต้น

การทดลองที่ 2: process ที่ดักจับ SIGTERM ได้

```
[sigdemo] pid = 5794, catching SIGTERM(15) and SIGINT(2)

$ kill 5794          # SIGTERM -> the handler runs, the process survives
[sigdemo] caught SIGTERM (15) -- handler ran, still alive

$ kill -INT 5794     # SIGINT -> the handler runs, the process survives
[sigdemo] caught SIGINT (2) -- handler ran, still alive

$ ps -o pid,stat,comm -p 5794
  PID STAT COMMAND
  5794 S+   q1_sigdemo      <-- still alive

$ kill -9 5794       # SIGKILL can be neither caught nor ignored
bash: 5794 Killed          /tmp/ps03_q1/q1_sigdemo
$ ps -o pid,stat,comm -p 5794
  PID STAT COMMAND
exit status of ps = 1
```

Code 19: SIGTERM ถูกดักจับ ต้องใช้ SIGKILL จึงจะจบได้

การทดลองที่ 3: SIGSTOP และ SIGCONT พักและปลุกแทนการฆ่า

```
$ ps -o pid,stat,comm -p 5803      # S = sleeping
  PID STAT COMMAND
  5803 S+   sleep
$ kill -STOP 5803
$ ps -o pid,stat,comm -p 5803      # T = stopped
  PID STAT COMMAND
  5803 T+   sleep
$ kill -CONT 5803
$ ps -o pid,stat,comm -p 5803      # back to S again
  PID STAT COMMAND
  5803 S+   sleep
```

Code 20: kill -STOP และ kill -CONT

การทดลองที่ 4: การฆ่า job ด้วย job spec %n

```
$ sleep 400 &
$ sleep 500 &
$ sleep 600 &
$ jobs -l
[1] 5813 Running          sleep 400 &
[2]- 5814 Running          sleep 500 &
[3]+ 5815 Running          sleep 600 &

$ kill %2                # send SIGTERM to job number 2
[2]- Terminated          sleep 500
$ jobs -l
[1]- 5813 Running          sleep 400 &
[3]+ 5815 Running          sleep 600 &

$ kill -9 %1 %3          # kill the two remaining jobs
bash: 5813 Killed          sleep 400
bash: 5815 Killed          sleep 600
$ jobs -l                # no output = no job left
```

Code 21: kill %n สำหรับ job ของ shell

การทดลองที่ 5: pgrep, pkill และ killall อ้างด้วยชื่อ

```
$ sleep 700 & sleep 700 & sleep 700 &
$ pgrep -a sleep
5822 sleep 700
5823 sleep 700
5824 sleep 700
$ pkill -f 'sleep 700'    # kill every process whose full command line matches
[1] Terminated          sleep 700
[2]- Terminated          sleep 700
[3]+ Terminated          sleep 700
$ pgrep -a sleep
exit status of pgrep = 1  (1 = nothing matched)

$ pgrep -a q1_sigdemo
5830 /tmp/ps03_q1/q1_sigdemo
5831 /tmp/ps03_q1/q1_sigdemo
$ killall -9 q1_sigdemo    # kill by exact program name
bash: 5830 Killed          /tmp/ps03_q1/q1_sigdemo > /dev/null
bash: 5831 Killed          /tmp/ps03_q1/q1_sigdemo > /dev/null
$ pgrep -a q1_sigdemo
exit status of pgrep = 1
```

Code 22: การฆ่า process ด้วยชื่อแทนการหา PID ที่ละตัว

การทดลองที่ 6: kill -0 ตรวจสอบว่า PID ยังอยู่หรือไม่

```
$ kill -0 5837    -> pid 5837 is alive (exit status 0)
$ kill 5837
$ kill -0 5837    -> bash: kill: (5837) - No such process (exit status 1)
```

Code 23: kill -0 ใช้ตรวจสอบโดยไม่ส่งสัญญาณจริง

1.3.5 อภิปรายผล

- **SIGTERM คือการ “ขอ” ส่วน SIGKILL คือการ “บังคับ”:** การทดลองที่ 2 แสดงให้เห็นชัดว่า kill ธรรมดาส่ง SIGTERM ซึ่งโปรแกรมมีสิทธิ์ดักจับและตัดสินใจเองว่าจะทำอะไร q1_sigdemo เลือกที่จะพิมพ์ข้อความแล้วอยู่ต่อ ทำให้คำสั่ง kill และ kill -INT ไม่มีผลใด ๆ ต่อการมีอยู่ของ process จนต้องใช้ kill -9 ซึ่งเคอร์เนลเป็นผู้จัดการให้เอง โปรแกรมไม่มีโอกาสแม้แต่จะรู้ตัว
- **ควรใช้ kill ก่อนเสมอ และใช้ kill -9 เป็นทางเลือกสุดท้าย:** เหตุผลคือ SIGTERM เปิดโอกาสให้โปรแกรมทำ cleanup ก่อนจบ เช่น เขียนข้อมูลใน buffer ลงไฟล์ ปิด socket ลบไฟล์ชั่วคราว หรือปลด lock ส่วน SIGKILL ตัดจบทันที ทำให้ไฟล์ชั่วคราวค้าง ฐานข้อมูลอาจไม่ถูก commit และ process ลูกอาจกลายเป็น orphan ตามที่ได้ศึกษาใน PS02 ในทางปฏิบัติควรลอง kill PID แล้วรอสัก 5-10 วินาที ถ้ายังไม่ตายจึงค่อยใช้ kill -9 PID
- **SIGSTOP ไม่ใช้การฆ่า และไม่ใช้การ “หยุดถาวร”:** การทดลองที่ 3 แสดงว่า process เปลี่ยนสถานะจาก S เป็น T แล้วกลับมาเป็น S ได้ด้วย SIGCONT ในระหว่างที่เป็น T process ยังคงอยู่ในระบบและยังจองหน่วยความจำอยู่เหมือนเดิม เพียงแต่ scheduler จะไม่จัด CPU ให้อีกจนกว่าจะได้ SIGCONT ประโยชน์จริงคือใช้ “แช่แข็ง” งานหนักไว้ชั่วคราวเมื่อต้องการ CPU ไปทำอย่างอื่นก่อน แล้วค่อยปลุกกลับมาโดยไม่ต้องเริ่มงานใหม่ทั้งหมด (สัญญาณคูนีคือกลไกเบื้องหลังคำสั่ง CTRL+Z, fg, bg ซึ่งจะทดลองต่อในข้อ 2)
- **%n เป็นของ shell ไม่ใช่ของเคอร์เนล:** เคอร์เนลรู้จักแค่ PID เท่านั้น เลข job [1] [2] [3] เป็นสิ่งที่ shell จัดไว้เองสำหรับ process ที่ตัวเองสร้าง เวลาสั่ง kill %2 shell จะแปลง %2 เป็น PID (หรือ process group) แล้วค่อยส่งสัญญาณ ผลที่ตามมาคือ job spec ใช้ได้เฉพาะใน shell ที่เป็นเจ้าของ job นั้น ถ้าเปิด terminal ใหม่แล้วสั่ง kill %1 จะไม่พบ job ต้องกลับไปใช้ PID แทน ข้อดีของ %n คือสะดวกและอ้างได้หลายตัว พร้อมกันดังที่ทดลองด้วย kill -9 %1 %3
- **pkill -f สะดวกแต่ต้องระวังมาก:** option -f สั่งให้จับคู่กับ command line เต็ม ไม่ใช่แค่ชื่อ โปรแกรม ซึ่งกว้างกว่าที่คิดมาก ระหว่างการเก็บผลการทดลองนี้เกิดปัญหาจริง คือสคริปต์ที่สั่ง pkill -f 'sleep 700' ได้ฆ่า ตัวเอง ไปด้วย เพราะ command line ของสคริปต์นั้นมีข้อความ sleep 700 ประกอบอยู่ จึงเข้าเงื่อนไขไปด้วย ทำให้ผลการทดลองขาดหายไปครึ่งหนึ่ง แนวปฏิบัติที่ปลอดภัยคือ สั่ง pgrep ด้วยเงื่อนไขเดียวกันก่อน เพื่อดูรายชื่อเป้าหมาย แล้วจึงเปลี่ยนเป็น pkill เมื่อมั่นใจแล้ว ซึ่งเป็นลำดับที่ใช้ในการทดลองที่ 5
- **killall ปลอดภัยกว่าแต่แคบกว่า:** killall จับคู่กับ ชื่อโปรแกรม แบบตรงตัว จึงไม่พลาดไปโดน คำสั่งอื่นที่บังเอิญมีข้อความคล้ายกันเหมือน pkill -f แต่ก็แลกมาด้วยความยืดหยุ่นที่น้อยกว่า และต้องระวังว่าบนระบบ UNIX บางตระกูล (เช่น Solaris) คำสั่งชื่อเดียวกันนี้หมายถึงการฆ่า ทุก process ในระบบ ซึ่งอันตรายมาก
- **kill -0 เป็นเครื่องมือตรวจสอบ ไม่ใช่การฆ่า:** เป็นสำนวนมาตรฐานที่ใช้ในสคริปต์เพื่อถามว่า PID ยังอยู่หรือไม่ โดยไม่ก่อผลข้างเคียง ค่า exit status 0 แปลว่ายังอยู่และเรามีสิทธิ์ส่ง สัญญาณ ส่วน 1 แปลว่าไม่พบ process นั้นแล้ว มักใช้คู่กับการรอให้ process จบหลังส่ง SIGTERM ก่อนตัดสินใจว่าจะต้องใช้ SIGKILL หรือไม่
- **เรื่องสิทธิ์:** ผู้ใช้ทั่วไปส่งสัญญาณได้เฉพาะ process ของตัวเอง การพยายามส่งให้ process ของผู้อื่น หรือของ root จะได้ Operation not permitted ต้องใช้ sudo เท่านั้น และแม้แต่ root ก็ฆ่า process

ที่ติดอยู่ในสถานะ D (uninterruptible sleep เช่นค้างรอ I/O) ด้วย `kill -9` ไม่ได้ เพราะสัญญาณจะถูกส่งไปรอไว้เฉย ๆ จนกว่า process จะออกจากสถานะนั้น

1.4 สรุปข้อ 1

Table 5: สรุปคำสั่งทั้งหมดที่ใช้ในข้อ 1

คำสั่ง	หน้าที่และจุดที่ต้องจำ
ps	ภาพนิ่งครั้งเดียว จบพื้นที่ เหมาะกับการค้นหาและการนำผลไปใช้ต่อในสคริปต์
ps aux	ดูทุก process แบบ BSD ให้ %CPU, %MEM, STAT
ps -ef	ดูทุก process แบบ UNIX ให้คอลัมน์ PPID
ps -eo ... --sort	เลือกคอลัมน์และเรียงลำดับเอง ยืดหยุ่นที่สุด
top	เฝ้าดูแบบเรียลไทม์ รีเฟรชเรื่อย ๆ มีสรุปภาพรวมของทั้งระบบและสั่งงานได้ในตัว
top -b -n 1	batch mode ถ่ายภาพครั้งเดียวเพื่อเก็บลงไฟล์หรือรายงาน
nice -n N cmd	เริ่มโปรแกรมด้วยค่า nice ที่กำหนด (เพิ่มค่าสะสม ไม่ใช่กำหนดตายตัว)
renice -n N -p PID	เปลี่ยนค่า nice ของ process ที่รันอยู่แล้ว ผู้ใช้ทั่วไปเพิ่มค่าได้อย่างเดียว การลดค่าต้องเป็น root
kill PID	ส่ง SIGTERM ให้โอกาสโปรแกรมเก็บกวาดก่อนจบ (ควรใช้ก่อนเสมอ)
kill -9 PID	ส่ง SIGKILL ดักจับไม่ได้ ใช้เป็นทางเลือกสุดท้าย
kill -STOP / -CONT	พักและปลุก process โดยไม่ทำลายทิ้ง
kill %n	อ้างด้วยเลข job ใช้ได้เฉพาะใน shell ที่เป็นเจ้าของ job
kill -0 PID	ตรวจว่า PID ยังอยู่หรือไม่ โดยไม่ส่งสัญญาณจริง
pgrep -a pattern	ค้นหา PID จากชื่อหรือ command line (ควรสั่งก่อน pkill เสมอ)
pkill -f pattern	ฆ่าจาก command line เต็ม สะดวกแต่กว้างและอันตราย
killall progname	ฆ่าจากชื่อโปรแกรมแบบตรงตัว แคบกว่าแต่ปลอดภัยกว่า

สิ่งที่ได้เรียนรู้จากข้อนี้และจะนำไปใช้ในข้อถัดไปมีสามเรื่องหลัก

1. ps กับ top ไม่ได้แทนกัน แต่ใช้คู่กัน top ใช้ “มองหา” ว่าปัญหาอยู่ที่ไหน แล้ว ps -o ใช้ “เจาะดู” เฉพาะ process ที่สนใจพร้อมคอลัมน์ที่ต้องการ
2. nice เป็นเพียงคำแนะนำต่อ scheduler ไม่ใช่การจองทรัพยากร และจะมีผลก็ต่อเมื่อ CPU ไม่พอใช้เท่านั้น การทดลองที่ได้ค่า 64 เท่า เกิดขึ้นได้เพราะบังคับให้แย่ง CPU แกนเดียวกันด้วย taskset
3. kill คือการ “ส่งสัญญาณ” ความเข้าใจเรื่องสัญญาณจะถูกใช้ต่อ โดยตรงในข้อ 2 เพราะการกด CTRL+Z คือการส่ง SIGTSTP คำสั่ง bg คือการส่ง SIGCONT และคำสั่ง fg คือการส่ง SIGCONT พร้อมกับคืนการควบคุม terminal ให้ job นั้น